

Requisitos, Gestão de Configurações e Controlo de Versões

Curso: LETI/LEI/LEI-PL

2026/2027

Objetivos

Esta ficha laboratorial tem como objetivo principal a introdução à plataforma de gestão de configurações **GitHub**, onde será criada um “*product backlog*” (Scrum) e que suporta o sistema de controlo de versões **Git**, em articulação com um IDE, que aqui será o IDE **IntelliJ IDEA Ultimate**. Pretende-se em particular que os alunos:

- Criem uma especificação de requisitos alinhada com a metodologia Scrum.
- Compreendam os conceitos de gestão de configurações e versões de software.
- Aprendam a gerir projetos colaborativos no GitHub em grupos de 3 a 4 alunos.
- Apliquem boas práticas de Engenharia de Software: product backlog ágil, documentação com Javadoc, *pull requests*, automação com GitHub Actions.
- Experimentem diferentes interfaces de interação com o Git: plataforma web, IDE e linha de comandos.
- Explorem a utilização de LLMs como auxiliares de desenvolvimento.

Nota: Provavelmente esta ficha laboratorial não será concluída integralmente nas 2 aulas alocadas para o efeito. Nesse caso, o trabalho deve continuar autonomamente fora das aulas e as dúvidas que possam surgir serão esclarecidas nos horários de atendimento anunciados, ou acordados.

Instrumentos de aprendizagem



GitHub: plataforma para gestão colaborativa de requisitos e código com Git



Git: sistema de controlo de versões distribuído



IntelliJ IDEA Ultimate: IDE profissional

Informação de apoio

- [Slides das aulas teóricas no Moodle](#).
- [Livro Pro Git](#), de Scott Chacon e Ben Straub.
- Documentação oficial do [GitHub](#) e do [GitHub Actions](#) em particular.
- Documentação IntelliJ IDEA Ultimate relativa ao [GitHub](#) e ao [Git](#).
- Vídeos do GitHub [GitHub for Beginners](#) e [Getting started with GitHub Copilot](#).

Parte 1 - GitHub (via plataforma web)

A. Caso de estudo

Neste e noutros guiões vamos usar como projeto um conhecido jogo, a **Batalha Naval**, onde cada jogador começa por construir duas grelhas quadriculadas iguais de 10x10 quadrados. Uma das grelhas representa “o seu mar”, a outra “o mar do adversário”. Seguidamente, deverá posicionar os seus navios na sua grelha, seguindo a orientação horizontal ou vertical, sem que o adversário os veja. O número de navios é igual para cada jogador e deverão ser posicionados sem que possam tocar-se entre si, embora possam estar encostados à borda das grelhas, como no exemplo da Figura 1.

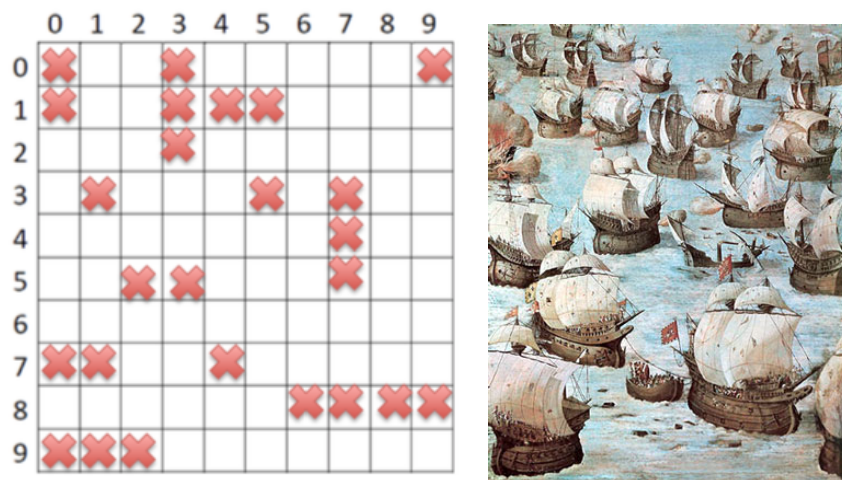


Figura 1: Exemplo de frota no jogo da Batalha Naval e [exemplo histórico](#)

Vamos jogar uma versão do tempo das Descobertas (*Discoveries Battleship Game*), em que usaremos nomes de navios desse tempo. A correspondência de nomes com aqueles que provavelmente conhece da batalha naval da atualidade é a que aparece na Tabela 1.

Tabela 1: Correspondência entre navios atuais e os da versão da época dos Descobrimentos

Batalha Naval	Descobrimentos	English	Dimensão	#Navios
Porta-aviões	Galeão	Galleon	5	1
Navio de 4 canhões	Fragata	Frigate	4	1
Navio de 3 canhões	Nau	Carrack	3	2
Navio de 2 canhões	Caravela	Caravel	2	3
Submarino	Barca	Barge	1	4

B. Criação de conta GitHub

1. Crie uma conta pública no GitHub com o seguinte formato: **LEI-XXXXX** ou **LETI-XXXXX**, em que XXXXX é o seu número de aluno. Personalize o seu perfil do GitHub com o primeiro nome e apelido e coloque a sua fotografia do Fenix+ ou LinkedIn. A correta identificação da conta do GitHub é muito importante, porque toda a avaliação do trabalho dos guiões é feita individualmente com base nas atividades que ficam registadas com carimbo temporal no GitHub.
2. Ative a autenticação de dois fatores (2FA) usando uma **passkey**.

C. Formação de grupos e repositório base

1. Forme um grupo de 3 ou 4 alunos.
2. O(A) aluno(a) com o número mais baixo deve [importar](#), na sua conta do GitHub o projeto disponibilizado em [projeto Battleship](#), como repositório público, dando-lhe o nome “*DiscoveriesBattleshipGame*”. Não sabe como se faz a importação no GitHub? Pergunte ao seu LLM preferido!
3. Esse(a) aluno(a) deve adicionar nesse repositório, como colaboradores, os outros colegas de grupo em *Settings* → *Collaborators*. Os convites respetivos aparecerão na área de notificações dos colegas, que aí deverão aceitar, clicando no convite, passando o projeto a estar visível na sua área de repositórios.

D. Exploração inicial do repositório

1. Na opção *Settings* → *General* confirme que está no ramo por omissão (*main*), nome habitual do ramo principal em repositórios Git.
2. Na opção *Insights* → *Network* veja um grafo com a história do projeto e identifique qual é o ramo com a atualização mais recente.
3. No ramo *main* atualize o ficheiro `README.md` usando —*Markdown*, que tem [uma sintaxe simples](#), indicando um nickname para o grupo e listando o curso, número e nome dos seus membros. Sugiro que crie uma tabela com ajuda de um LLM. Guarde as alterações com um commit. Volte ao grafo e constate que o seu commit lá aparece registado.
4. Todos os membros da equipa devem fazer alterações no `README.md`, nomeadamente adicionando-lhe várias seções, uma descrevendo os tipos de navios como anteriormente indicado neste guião, outra com as regras do jogo (ver à frente) e, se quiserem, outra com links para a [Wikipedia](#) sobre os navios existentes no tempo dos Descobrimentos, fotografias dos mesmos, etc. Pode ver um exemplo detalhado [aqui](#) para efeitos de inspiração e pode encontrar *cheatsheets* da sintaxe de *markdown* [aqui](#) ou [aqui](#).
5. Consulte a lista de ramos na opção *Code* → *Branches*. No ramo em que fez a atualização do `README` clique em “*Compare & pull request*” (o botão aparece se esse ramo estiver à frente do *main* ou de outro ramo). Escolha o “*main*” como ramo de destino e abra um “*Pull Request*”.
6. Todos os membros da equipa vão ver os pull requests pendentes na opção com esse nome no menu principal. Se não houver conflitos, outro membro da equipa, que não aquele que submeteu o pull request, verá o botão “*Merge pull request*” → “*Confirm merge*”.
7. Veja o que aconteceu no grafo e, na lista de ramos, procure o pull request que foi processado e, clicando no ícone correspondente, comente o que viu no grafo, isto é, o que resultou do pull request e do merge.

E. Especificação de requisitos / criação de um “Scrum Backlog”

1. Veja os exemplos de user stories fornecidos no Moodle.
2. Relembremos as funcionalidades de um jogo de batalha naval. Depois dos navios terem sido posicionados, cada jogador, à vez, “atira três tiros” sobre a frota adversária, referindo as respectivas coordenadas dos tiros (linha, coluna). O adversário deve referir o resultado dessa rajada de três tiros, informando se acertou em um ou mais navios e de que tipo, bem como os tiros na água! Cada jogador vai registrando na grelha do oponente os resultados dos seus tiros, identificando os navios afundados. Ganha o jogo o primeiro que atingir todos os navios da frota adversária. Pode experimentar jogar online [aqui](#), mas atenção que neste exemplo a forma e número de navios não são os que tradicionalmente usamos em Portugal.
3. Considere que é o *Product Owner* dessa app. Em seu entender, que requisitos adicionais lhe faltam? Por outras palavras, qual deverá ser o *Product Backlog*?
4. Crie colaborativamente com os colegas do grupo o *Product Backlog* no separador Issues do GitHub, redigindo *user stories* em Scrum, com base na exploração interativa do jogo de exemplo que experimentou atrás. Vai ter, pelo menos, de colocar a sua frota no tabuleiro (grelha), dar tiros sobre o tabuleiro do adversário, visualizar o seu tabuleiro e o que vai conhecendo do tabuleiro do seu oponente, guardando as jogadas numa base de dados, por exemplo, em JSON. Os grupos mais ousados vão querer jogar contra a IA, usando um modelo *open-source* obtido do site [Hugging Face](#).
5. Vamos agora usar a funcionalidade de automação da plataforma, o chamado *GitHub Actions*. Comece por ler este [Quickstart](#). Carregue agora a partir do Moodle o ficheiro *labels.json* (definição de novas etiquetas para os *Issues*) para a diretoria *.github* e o ficheiro *labels.yml* (script YAML para executar no GitHub Actions) para a diretoria *.github/workflows*. Na opção *Actions* do menu do GitHub, corra agora o script *Sync Labels* que está contido no ficheiro YAML anteriormente carregado. Use a etiqueta (*tag*) “ENHANCEMENT” na criação de *Issues* para cada *user story* e atribua-lhes uma prioridade com outra etiqueta, entre as cinco disponíveis para esse efeito (CRITICAL, HIGH, MEDIUM, LOW, REVISIT).
6. Imagine agora que está na reunião diária do Scrum em que tem de escolher uma nova user story para si próprio. O que fazer no GitHub? E quando noutro dia posterior quiser indicar que essa user story já foi implementada? E quando o Product Owner desiste de uma user story, como podemos indicar isso no GitHub? Responda a estas questões através de comentários no GitHub, junto dos próprios “issues”.

F. Colaboração avançada

1. Estude como se documentam programas Java usando Javadoc. A [página de ajuda do IntelliJ IDEA](#) é bastante boa, mas também pode perguntar ao seu LLM preferido como se faz.
2. Divida as classes do jogo entre os membros da equipa, para que cada membro documente uma classe com Javadoc, submetendo o código resultante via Pull Request para um colega rever. Nota: os LLMs como o ChatGPT, Copilot ou Claude Code são muito bons em gerar automaticamente comentários Javadoc.
3. Configure um workflow simples no GitHub Actions para notificar o colega responsável pelo Pull Request, no caso do GitHub não o fazer automaticamente.
4. Explore o menu Insights para analisar atividade, contribuições e rede de commits.

Parte 2 - Git no IntelliJ IDEA e em CLI

A. Instalação e configuração

1. Instale o IntelliJ IDEA Ultimate (licença académica gratuita via JetBrains Student Program).
2. Configure o plugin Git/GitHub.

B. Operações básicas no IDE

1. Faça o clone do repositório de grupo diretamente no IntelliJ.
2. Crie um ramo (branch) e mude para ele.
3. Nesse ramo, complemente a documentação anterior em Javadoc das classes do projeto, mas agora no IDE.
4. Crie no IDE as páginas html de documentação, colocando-as na diretoria *docs/* do projeto (na raiz, prática comum no GitHub) e consulte-as no seu browser.
5. Realize *commit* e *push* das alterações para que fiquem refletidas no GitHub. O Copilot também é bom para redigir mensagens de commit claras. Experimente!

C. Comparação crítica

1. Reflita: Quais as diferenças entre trabalhar via web e via IDE? Em que situações é preferível cada abordagem?

D. Comandos fundamentais do Git em CLI

1. Revise os conceitos de working area, staging area, local e remote repository.
2. Pratique os comandos: `git init`, `clone`, `add`, `commit`, `push`, `pull`.

E. Gestão de branches e conflitos

1. Crie, alterne e una branches (`branch`, `checkout`, `merge`, `rebase`).
2. Simule um conflito de merge e resolva-o manualmente.

F. Exploração avançada

1. Utilize `git stash` para guardar alterações temporárias.
2. Crie tags (`git tag`) para marcar versões.
3. Explore modelos de branching (Git Flow, GitHub Flow).

G. Desafios práticos

1. Em grupo, simulem alterações concorrentes no mesmo ficheiro.
2. Usem um para apoiar a resolução de conflitos e documentem o processo.
3. Publiquem uma release etiquetada com um tag.

Critérios de correção

O Guião será avaliado individualmente para cada membro do grupo, baseado nas evidências do trabalho realizado existentes nos repositórios do GitHub.

Os critérios de avaliação são os seguintes:

1. Issues - 20%.
2. Actions - 20%.
3. Pull Requests - 20%.
4. Branchs - 20%.
5. Commits - 20%.